

Práctica 13 — Angular + Formularios + API REST

(Semana 6: Estudiantes)

Objetivo

Construir una aplicación web en Angular que gestione estudiantes (nombre y email) con formulario reactivo, validación síncrona y asíncrona, y conexión a la API REST de la Semana 6. Esta guía permite desarrollarla, paso a paso.

1. Requisitos previos

- Node LTS y Angular CLI instalados.
- Editor de código (VS Code recomendado).
- API de Semana 6 en ejecución en <http://localhost:8080/api/estudiantes>.
- Entidad Java con campos: id, nombre, email (ambos obligatorios).
- Git opcional para control de versiones.

2. Crear el proyecto base

```
ng new gestion-estudiantes --style=scss --routing
cd gestion-estudiantes
```

Configura SASS en `src/styles/_variables.scss` y `src/styles/_mixins.scss`:

```
$primary: #3f51b5;
$danger: #e53935;
$radius: 10px;

@mixin form-field() {
  padding: .5rem .75rem;
  border: 1px solid #d1d5db;
  border-radius: $radius;
}
@mixin form-error() {
  font-size: .8rem;
  color: $danger;
  margin-top: .25rem;
}
```

3. Generar módulo y componentes

```
ng g module features/estudiantes --route estudiantes --module app.module
ng g component features/estudiantes/pages/lista-estudiantes
ng g component features/estudiantes/pages/form-estudiante
ng g service features/estudiantes/data/estudiantes
```

4. Modelo de datos

```
// src/app/features/estudiantes/data/estudiante.model.ts
export interface Estudiante {
  id?: number;
  nombre: string;
  email: string;
}
```

5. Servicio HTTP (con validación asincrónica de email)

```
// src/app/features/estudiantes/data/estudiantes.service.ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, of } from 'rxjs';
import { delay, map } from 'rxjs/operators';
import { Estudiante } from './estudiante.model';

@Injectable({ providedIn: 'root' })
export class EstudiantesService {
  private base = 'http://localhost:8080/api/estudiantes';

  constructor(private http: HttpClient) {}

  list(): Observable<Estudiante[]> { return this.http.get<Estudiante[]>(this.base); }
  get(id: number): Observable<Estudiante> { return
this.http.get<Estudiante>(`${this.base}/${id}`); }
  create(data: Estudiante): Observable<Estudiante> { return
this.http.post<Estudiante>(this.base, data); }
  update(id: number, data: Estudiante): Observable<Estudiante> { return
this.http.put<Estudiante>(`${this.base}/${id}`, data); }
  delete(id: number): Observable<void> { return
this.http.delete<void>(`${this.base}/${id}`); }

  // Simulación de validación asincrónica (puede adaptarse al backend real)
  existsEmail(email: string): Observable<boolean> {
    return this.list().pipe(
      delay(400),
      map(list => list.some(e => e.email.toLowerCase() === email.toLowerCase()))
    );
  }
}
```

6. Validadores personalizados

```
// src/app/shared/validators/email-unico.validator.ts
import { AbstractControl, AsyncValidatorFn } from '@angular/forms';
import { map } from 'rxjs/operators';
import { EstudiantesService } from
```

```
'../../features/estudiantes/data/estudiantes.service';

export function emailUnicoValidator(service: EstudiantesService): AsyncValidatorFn {
  return (control: AbstractControl) => {
    return service.existsEmail(control.value).pipe(
      map(existe => (existe ? { emailOcupado: true } : null))
    );
  };
}
```

7. Formulario Reactivo (Semana 13 — foco)

```
// src/app/features/estudiantes/pages/form-estudiante/form-estudiante.component.ts
import { Component, OnInit } from '@angular/core';
import { FormBuilder, Validators } from '@angular/forms';
import { ActivatedRoute, Router } from '@angular/router';
import { EstudiantesService } from '../../data/estudiantes.service';
import { emailUnicoValidator } from '../../shared/validators/email-unico.validator';
```

```
const SOLO_LETRAS = /^[A-Za-zÁÉÍÓÚÑáéíóúñ ]+$/;
```

```
@Component({
  selector: 'app-form-estudiante',
  templateUrl: './form-estudiante.component.html',
  styleUrls: ['./form-estudiante.component.scss']
})
export class FormEstudianteComponent implements OnInit {
  editMode = false;
  id: number | null = null;
  formError: string | null = null;

  form = this.fb.group({
    nombre: ['', [Validators.required, Validators.minLength(3),
    Validators.pattern(SOLO_LETRAS)]],
    email: ['', [Validators.required, Validators.email],
    [emailUnicoValidator(this.api)]]
  });

  constructor(
    private fb: FormBuilder,
    private api: EstudiantesService,
    private route: ActivatedRoute,
    private router: Router
  ) {}

  ngOnInit(): void {
    const idParam = this.route.snapshot.paramMap.get('id');
    if (idParam) {
      this.editMode = true;
      this.id = Number(idParam);
      this.api.get(this.id).subscribe({
```

```

        next: (res) => this.form.patchValue(res),
        error: () => this.formError = 'No se pudo cargar el estudiante.'
    });
}
}

isInvalid(ctrl: string): boolean {
    const c = this.form.controls[ctrl];
    return !(c && c.invalid && (c.dirty || c.touched));
}

onSubmit(): void {
    if (this.form.invalid) {
        this.form.markAllAsTouched();
        const firstInvalid = Object.keys(this.form.controls).find(k =>
this.form.controls[k].invalid);
        document.getElementById(firstInvalid || '').focus();
        return;
    }

    const estudiante = this.form.value;
    if (this.editMode && this.id) {
        this.api.update(this.id, { id: this.id, ...estudiante }).subscribe({
            next: () => this.router.navigate(['/estudiantes']),
            error: () => this.formError = 'No se pudo actualizar el estudiante.'
        });
    } else {
        this.api.create(estudiante).subscribe({
            next: () => this.router.navigate(['/estudiantes']),
            error: () => this.formError = 'No se pudo registrar el estudiante.'
        });
    }
}
}
}

```

```

// src/app/features/estudiantes/pages/form-estudiante/form-estudiante.component.html
<section class="page">
    <h2>{{ editMode ? 'Editar Estudiante' : 'Registrar Estudiante' }}</h2>

    <div *ngIf="formError" class="alert alert-error">{{ formError }}</div>

    <form [formGroup]="form" (ngSubmit)="onSubmit()" novalidate>
        <div class="field">
            <label for="nombre">Nombre</label>
            <input id="nombre" formControlName="nombre" [class.invalid]="isInvalid('nombre')"/>
        </div>

        <div class="error" *ngIf="isInvalid('nombre')">
            <small *ngIf="form.controls['nombre'].errors?.['required']">Obligatorio</small>
            <small *ngIf="form.controls['nombre'].errors?.['minlength']">Mínimo 3
caracteres</small>
            <small *ngIf="form.controls['nombre'].errors?.['pattern']">Solo letras y

```

```

    espacios</small>
  </div>
</div>

  <div class="field">
    <label for="email">Email</label>
    <input id="email" formControlName="email" type="email"
[class.invalid]="isInvalid('email')" />
    <div class="error" *ngIf="isInvalid('email')">
      <small *ngIf="form.controls['email'].errors?.['required']">Obligatorio</small>
      <small *ngIf="form.controls['email'].errors?.['email']">Formato
inválido</small>
      <small *ngIf="form.controls['email'].errors?.['emailOcupado']">Email ya
registrado</small>
    </div>
  </div>

  <div class="actions">
    <button type="submit" [disabled]="form.invalid || form.pending">{{ editMode ?
'Actualizar' : 'Guardar' }}</button>
    <a routerLink="/estudiantes" class="btn btn-link">Cancelar</a>
  </div>
</form>
</section>

```

8. Listado de estudiantes

```

<!-- src/app/features/estudiantes/pages/lista-estudiantes/lista-
estudiantes.component.html -->
<section class="page">
  <header class="page-header">
    <h2>Estudiantes</h2>
    <a routerLink="/estudiantes/nuevo" class="btn btn-primary">Nuevo</a>
  </header>

  <table class="tabla">
    <thead><tr><th>ID</th><th>Nombre</th><th>Email</th><th></th></tr></thead>
    <tbody>
      <tr *ngFor="let e of data">
        <td>{{e.id}}</td>
        <td>{{e.nombre}}</td>
        <td>{{e.email}}</td>
        <td><a [routerLink]="['/estudiantes', e.id, 'editar']">Editar</a></td>
      </tr>
    </tbody>
  </table>
</section>

```

9. Autoevaluación

- Formulario reactivo con validaciones síncronas y asíncronas.

- Mensajes de error claros.
- Integración funcional con API REST Semana 6.
- CRUD completo (listar, crear, actualizar, eliminar opcional).
- Estilos SASS aplicados.